

Applying Model Context Protocol for Offline Small Language Models in Industrial Data Management [†]

Nian-Ze Hu , You-Xin Lin *, Hao-Lun Huang, Po-Han Lu, Chih-Chen Lin, Yu-Tzu Hung, Sing-Cih Jhang and Pei-Yu Chou

Smart Machinery and Intelligent Manufacturing Research Center, National Formosa University, Wunhua Rd., Huwei Township, Yunlin County 632, Taiwan; drhu@nfu.edu.tw (N.-Z.H.); 11261108@nfu.edu.tw (H.-L.H.); 11361132@nfu.edu.tw (P.-H.L.); 11361115@nfu.edu.tw (C.-C.L.); 11461104@nfu.edu.tw (Y.-T.H.); 11461102@nfu.edu.tw (S.-C.J.); 11461129@nfu.edu.tw (P.-Y.C.)

* Correspondence: 11261304@nfu.edu.tw

[†] Presented at the 7th Eurasia Conference on IoT, Communication and Engineering 2025 (ECICE 2025), Yunlin, Taiwan, 14–16 November 2025.

Abstract

In recent years, Large Language Models (LLMs) have demonstrated strong capabilities in contextual reasoning and knowledge retrieval. However, their application in industrial domains is limited by concerns regarding data security, reliance on cloud infrastructure, and high operational costs. To address these challenges, this study proposes the use of the Model Context Protocol (MCP) as a middleware framework that enables the deployment of offline-operable Small Language Models (SLMs) for industrial data processing. MCP facilitates structured interaction between SLMs and external resources (e.g., databases, APIs, and processors), allowing secure and controlled data access without exposing proprietary systems. As illustrated in the proposed framework, user input is first processed by the SLM (Qwen-7B) for intent determination. When external data is required, MCP coordinates the invocation of relevant resources and integrates the returned results into the model. The SLM then generates the final response. This approach enables SLMs to perform local computation for contextual analysis and decision support while maintaining low computational requirements and full data locality. The proposed system eliminates dependence on cloud-based LLM services and enhances security and cost efficiency. Experimental results demonstrate that the MCP-based architecture provides a practical and effective solution for deploying intelligent assistants in industrial environments without relying on large-scale external AI services.

Keywords: model context protocol (MCP); small language models (SLMs); offline industrial AI; manufacturing data management; secure and cost-efficient computing; smart factory/Industry 4.0



Academic Editors: Teen-Hang Meen, Chi-Ting Ho and Cheng-Fu Yang

Published: 7 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Many companies have regarded language models as essential tools for improving work efficiency [1–3]. However, mainstream high-performance large language models (LLMs), which contain billions to tens of billions of parameters, require substantial computational resources and hardware costs for training and inference due to their enormous scale [1,4]. Moreover, their reliance on external services can also raise cybersecurity concerns, such as the potential leakage of corporate confidential information [3].

In contrast, small language models (SLMs), with their smaller parameter sizes, are easier for small and medium-sized enterprises to deploy and can significantly reduce costs [5]. However, their response accuracy and flexibility are often limited [1]. To address this issue, this study introduces the model context protocol (MCP) to enhance the response capability of SLMs through targeted customization [3,5]. This approach not only reduces dependency on networks and expensive hardware but also enhances cost-effectiveness while maintaining information security [3].

2. Materials and Methods

This study aims to explore how MCP is used to enhance the application performance of SLMs, with Qwen-7B serving as the core model. The research process includes model selection, MCP integration, data processing, as well as experimental design and evaluation.

2.1. Research Framework

The research was conducted in four main stages: (1) model selection, (2) mcp integration, (3) data processing, and (4) experimental design and evaluation. This process ensures that the study validates the enhancement effects of MCP on Traditional Chinese SLMs while balancing both cost efficiency and information security.

2.2. SLM: Qwen-7B

We used Qwen-7B as an SLM for the following reasons.

- Native traditional Chinese support: It directly processes texts and instructions within Taiwanese and Mandarin Chinese contexts.
- Moderate scale (7 billion parameters): It requires lower computational resources, making it suitable for small and medium-sized enterprise environments.
- Local deployment: It runs on graphic processing unit (GPU) servers without relying on cloud services, thereby reducing cybersecurity risks.

2.3. MCP Integration

Through the use of MCP, we integrate Qwen-7B with external tools and resources. The main functions include a prompt template design to establish context prompts tailored for Traditional Chinese, tool functions (tools) to support SQL 2019 queries, data transformation, and document retrieval, and resource access (resources) to encapsulate SQL Server and industrial data as MCP resources.

As shown in Figure 1, user input is first processed by the SLM (Qwen-7B) to determine intent. If external data is required, the MCP tool coordination center is responsible for invoking the corresponding external resources (such as databases, APIs, or processors) and integrating the returned results back into the SLM. The model then generates the final response. If no external data is needed, the SLM directly outputs the response to the user.

The data used in this study include the simulated data, the traditional Chinese NLPtest corpus, and the resource of SQL server and industrial data encapsulated as MCP resources. The experimental environment of this study is configured as follows:

- Development environment: Python 3.10.9 + FastMCP (v0.1, 2024);
- Hardware: A single server equipped with a 12GB GPU;
- Database: SQL Server.

To evaluate the enhancement effects of MCP on Qwen-7B, we adopt the following metrics: response accuracy, efficiency (latency), and stability.

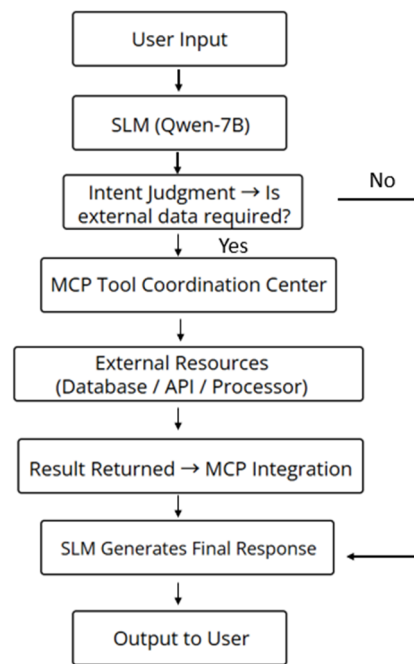


Figure 1. Process of MCP integration.

3. Results

We use Qwen-7B as the core SLM and integrate it through MCP to evaluate its feasibility and performance differences in data querying and external tool invocation. To illustrate the impact of MCP on the model's capabilities, this section presents actual interaction results and system response changes before and after its implementation.

Before the introduction of MCP, the model was limited to natural language semantic understanding. When a user input "Query all columns of CD1" (Figure 2), Qwen-7B provided a semantic explanation, indicating that additional data sources or structural descriptions (such as SQL tables or Excel files) were required to provide the column information. At this stage, the model lacked actual access to external databases and could only perform reasoning and guidance at the textual level.

Your question: Query all fields in CD1

Qwen's answer:

To help you query all fields in CD1, I need more background information. For example, which file or data table does CD1 belong to? Is it from a specific system or domain? If you can provide more detailed information, I can better provide you with the list of fields and related details you need.

If CD1 is a physical table or file, you can provide its original format (such as Excel or SQL table) or describe its structure. This way I can assist in listing the related fields for you.

Figure 2. User command in SLM with MCP.

After integrating MCP, the model interacts with external tools. When the user entered the same query command (Figure 3), the system triggered the MCP tool coordination mechanism, and the model automatically generated the following SQL query: `SELECT COLUMN_NAME, DATA_TYPE FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'CD1'`.

This action was executed using MCP's `query_db` tool, successfully returning the query results from the database, including the column names and corresponding data types in the CD1 table (e.g., `FEE_YM`: date, `HOSP_ID`: nvarchar, `TREAT_AMT`: int, etc.). This demonstrates that the model not only understands the query intent but can also autonomously invoke external resources through MCP to complete the task.

```

User: Query all columns of CD1
Processing...
Loading Qwen model...
torch_dtype is deprecated! Use dtype instead!
Loading checkpoint shards: 100% (4/4) [00:45<00:00, 11.33s/it]
Model loaded successfully!
Run tool: query_db
Execute SQL: SELECT COLUMN_NAME, DATA_TYPE FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'CD1'
Model: TOOL_CALL: {"tool": "query_db", "args": {"sql": " SELECT COLUMN_NAME, DATA_TYPE FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'CD1'"}}
Result:
01: {'COLUMN_NAME': 'FEE_YM', 'DATA_TYPE': 'date'}
02: {'COLUMN_NAME': 'APPL_TYPE', 'DATA_TYPE': 'tinyint'}
03: {'COLUMN_NAME': 'HOSP_ID', 'DATA_TYPE': 'nvarchar'}
04: {'COLUMN_NAME': 'APPL_DATE', 'DATA_TYPE': 'date'}
05: {'COLUMN_NAME': 'TREAT_AMT', 'DATA_TYPE': 'money'}
06: {'COLUMN_NAME': 'SEQ_NO', 'DATA_TYPE': 'int'}
07: {'COLUMN_NAME': 'CURE_ITEM_NO1', 'DATA_TYPE': 'nvarchar'}
08: {'COLUMN_NAME': 'CURE_ITEM_NO2', 'DATA_TYPE': 'tinyint'}
09: {'COLUMN_NAME': 'CURE_ITEM_NO3', 'DATA_TYPE': 'tinyint'}
10: {'COLUMN_NAME': 'CURE_ITEM_NO4', 'DATA_TYPE': 'tinyint'}
11: {'COLUMN_NAME': 'FUNC_TYPE', 'DATA_TYPE': 'nvarchar'}
12: {'COLUMN_NAME': 'FUNC_DATE', 'DATA_TYPE': 'date'}
13: {'COLUMN_NAME': 'TREAT_END_DATE', 'DATA_TYPE': 'date'}
14: {'COLUMN_NAME': 'ID_BIRTHDAY', 'DATA_TYPE': 'date'}
15: {'COLUMN_NAME': 'ID', 'DATA_TYPE': 'nvarchar'}
16: {'COLUMN_NAME': 'CARD_SEQ_NO', 'DATA_TYPE': 'nvarchar'}
17: {'COLUMN_NAME': 'GAVE_KIND', 'DATA_TYPE': 'tinyint'}
18: {'COLUMN_NAME': 'PART_NO', 'DATA_TYPE': 'nvarchar'}
19: {'COLUMN_NAME': 'ACODE_ICD9_1', 'DATA_TYPE': 'nvarchar'}
20: {'COLUMN_NAME': 'ACODE_ICD9_2', 'DATA_TYPE': 'int'}
21: {'COLUMN_NAME': 'ACODE_ICD9_3', 'DATA_TYPE': 'int'}
22: {'COLUMN_NAME': 'ICD_OP_CODE', 'DATA_TYPE': 'smallint'}
23: {'COLUMN_NAME': 'DRUG_DAY', 'DATA_TYPE': 'int'}
24: {'COLUMN_NAME': 'MED_TYPE', 'DATA_TYPE': 'tinyint'}
25: {'COLUMN_NAME': 'PRSN_ID', 'DATA_TYPE': 'nvarchar'}
26: {'COLUMN_NAME': 'PHAR_ID', 'DATA_TYPE': 'nvarchar'}
27: {'COLUMN_NAME': 'DRUG_AMT', 'DATA_TYPE': 'int'}
28: {'COLUMN_NAME': 'TREAT_CODE', 'DATA_TYPE': 'int'}
29: {'COLUMN_NAME': 'DIAG_AMT', 'DATA_TYPE': 'smallint'}
30: {'COLUMN_NAME': 'DSVC_NO', 'DATA_TYPE': 'nvarchar'}
31: {'COLUMN_NAME': 'DSVC_AMT', 'DATA_TYPE': 'int'}
32: {'COLUMN_NAME': 'BY_PASS_CODE', 'DATA_TYPE': 'nvarchar'}
33: {'COLUMN_NAME': 'T_AMT', 'DATA_TYPE': 'int'}
34: {'COLUMN_NAME': 'PART_AMT', 'DATA_TYPE': 'varchar'}
35: {'COLUMN_NAME': '[unreadable]', 'DATA_TYPE': 'smallint'}
36: {'COLUMN_NAME': '[unreadable]', 'DATA_TYPE': 'int'}
37: {'COLUMN_NAME': 'ID_SEX', 'DATA_TYPE': 'nvarchar'}

```

Figure 3. Query command to trigger the MCP tool coordination mechanism.

The introduction of MCP enables the following features and changes.

- Automated tool invocation: The model automatically determines, based on user input, whether external data queries using MCP are needed, creating a dynamic collaboration between semantic reasoning and tool execution.
- Data return integration: After query results are returned through MCP, the model can convert structured data (e.g., database columns) into natural language descriptions, providing more comprehensible explanations.
- Response traceability: All tool invocations are recorded in JSON format (e.g., TOOL_CALL and args blocks), ensuring transparency and reproducibility of the query process.
- Enhanced extensibility: The system can expand with new tools (e.g., APIs, file processing modules) according to task requirements without retraining the model.

Overall, after integrating MCP, Qwen-7B performs higher-level task processing in the same language interaction scenarios, transforming from simple language understanding to a complete closed-loop workflow of “semantic judgment→tool invocation→result integration.”

4. Discussion

MCP enables the capabilities of the SLM to expand. The results show that, after integrating MCP, Qwen-7B autonomously interprets query intent and executes database commands, demonstrating a significant behavioral transformation. This change indicates that MCP not only provides a bridge for external resource invocation but also enables the model to perform dynamic tool selection and contextual understanding.

4.1. Enhancement of Model Capability

Traditional SLMs are limited by their model parameters and training data, making them unable to directly connect to external data sources or tools. After integrating MCP, the model can autonomously perform tasks such as database queries, document retrieval, or data transformation through a “intent judgment → tool invocation → result integration” workflow. This architecture effectively compensates for the shortcomings of small models in knowledge updating and data access, enabling them to maintain high information timeliness and task orientation even in local environments.

In this study, Qwen-7B executes SQL queries via MCP without retraining. This expansion approach demonstrates that SLMs can achieve application flexibility comparable to large models through a structured invocation mechanism at the protocol layer.

4.2. Security and Deployment Considerations

The integration of MCP improves data security and deployment flexibility. Since all query processes are executed on local servers, data does not need to be uploaded to the cloud or external model providers, effectively reducing the risk of sensitive information leakage. In addition, the tool invocation workflow is recorded in a JSON structure, ensuring traceability and auditability, which aligns with internal corporate compliance and cybersecurity requirements. Compared with large cloud-based language models (LLMs), this SLM + MCP architecture is better suited for deployment in small and medium-sized enterprises, balancing both performance and privacy.

4.3. Practical Implications

The results of this study indicate that SLMs do not necessarily require massive parameters to accomplish complex tasks. Through the functional extension provided by MCP, small models can focus on semantic reasoning and decision-making while delegating data access and computation to external modules, forming a modular intelligent agent. This architecture demonstrates practical application potential in areas such as data querying, industrial process monitoring, educational assistance, and document retrieval.

4.4. Limitations and Future Work

Although the integration of MCP significantly enhances the practical capabilities of the model, several limitations remain. First, the tool selection and invocation logic in MCP currently rely on predefined rules and lack self-learning capabilities. Second, the system’s performance scheduling and resource management during parallel tool execution still require optimization. Future research could explore automated tool selection strategies, such as reinforcement learning-based decision mechanisms and test the integration performance with databases from different domains. Furthermore, combining MCP with semantic memory could enable SLMs to acquire continuous learning and contextual memory capabilities during task execution.

5. Conclusions

We integrated SLM and Qwen-7B and investigated the feasibility of enhancing the model’s external interaction capabilities through MCP. The results show that, after integrating MCP, Qwen-7B automatically generates and executes database query commands in response to user natural language queries, performing data retrieval and result integration. This demonstrates a behavioral shift from simple semantic understanding to task execution capability. The introduction of MCP expands the application scope of the SLM but also addresses the limitations of traditional language models in knowledge timeliness and external data access. This architecture enhances system security and controllability, allowing

the model to operate on local servers to prevent sensitive corporate data leakage while enabling modular integration of various data sources and tools.

The SLM + MCP integration offers the following advantages.

- Functional extensibility: It enables SLMs to autonomously invoke external tools and data resources through the protocol layer.
- Security and local deployment: It reduces reliance on the cloud while enhancing data protection.
- Reusable and scalable architecture: It allows rapid integration of different modules, such as databases, APIs, or computing nodes, based on application needs.
- Practical application potential: It applies to smart factories, educational systems, knowledge retrieval, and automated report generation.

Future research is necessary to explore MCP's capabilities in multi-tool collaboration, task decomposition, and semantic memory integration to realize intelligent agent systems with autonomous learning and dynamic scheduling features.

Author Contributions: Conceptualization, N.-Z.H. and Y.-X.L.; methodology, Y.-X.L.; software, H.-L.H., P.-H.L. and C.-C.L.; validation, Y.-X.L.; formal analysis, Y.-X.L.; investigation, H.-L.H., P.-H.L. and C.-C.L.; resources, Y.-T.H., S.-C.J. and P.-Y.C.; writing—original draft preparation, Y.-X.L.; writing—review and editing, N.-Z.H.; supervision, N.-Z.H. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not Applicable.

Informed Consent Statement: Not Applicable.

Data Availability Statement: The data used in this study are available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Zhao, W.X.; Zhou, K.; Li, J.; Tang, T.; Wang, X.; Hou, Y.; Min, Y.; Zhang, B.; Zhang, J.; Dong, Z.; et al. A Survey of Large Language Models. *arXiv* **2023**, arXiv:2303.18223. [CrossRef] [PubMed]
2. Touvron, H.; Martin, L.; Stone, K.; Albert, P.; Almahairi, A.; Babaei, Y.; Bashlykov, N.; Batra, S.; Bhargava, P.; Bhosale, S.; et al. LLaMA 2: Open Foundation and Fine-Tuned Chat Models. *arXiv* **2023**, arXiv:2307.09288. [CrossRef]
3. Hou, X.; Zhao, Y.; Wang, H. LLM Applications: Current Paradigms and the Next Frontier. *arXiv* **2025**, arXiv:2503.04596. [CrossRef]
4. Yang, A.; Yang, B.; Hui, B.; Zheng, B.; Yu, B.; Zhou, C.; Li, C.; Li, C.; Liu, D.; Huang, F.; et al. Qwen2 Technical Report. *arXiv* **2024**, arXiv:2407.10671. [CrossRef]
5. OpenAI. Model Context Protocol (MCP) Specification. Available online: <https://platform.openai.com/docs> (accessed on 12 September 2025).

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.